ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Nature of Assignment: Individual

Duration: 1 Week

Submission Date: 09.11.26

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at

K5 level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/vaishu1104/DSA-0118-

192524143-Assignment1.gitHow to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the

C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time
complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least
Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places.

The number of journeys processed may grow:

1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day)

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks

Bloom's Level: K5 – Evaluate

Analyse the fare problem from the object-oriented point of view.

Discuss:

• The objects in the problem (card, passenger, journey, zone, fare table) and the data and behaviour of each

• How the same problem would be written in a procedural style, and what changes in an object-oriented style

• The basic principles of OOP – encapsulation, abstraction, inheritance and polymorphism

– and where each could appear in this system

• The rules every correct journey must satisfy (for example: balance never below Rs 50 after a journey)

You are expected to justify each object and rule, not merely list names.

Alternative Concept and Complexity Analysis

If the fare system were written in a purely procedural style (global variables and free functions) instead of SmartCard, Journey and FareTable classes, what would the solution be? Analyse whether the time per journey or the memory per card changes (both are O(1)), and compare how much code must be changed when one fare rule changes.

Evidence Expected

A clear object-oriented analysis with objects, their data and behaviour, rules and stated assumptions.

ANS:

Task01 – Understand the problem and identify objects

Aim: To implement and demonstrate the task using a simple C++ program.

Complete code

```
#include <iostream>
#include <string>
using namespace std;

class SmartCard {
private:
    string cardId;
    long long balancePaise;

public:
    SmartCard(string id, long long balance) {
        cardId = id;
        balancePaise = balance;
    }

    bool canTravel(long long farePaise) {
        return farePaise >= 0 && balancePaise - farePaise >= 5000;
    }

    void deductFare(long long farePaise) {
        if (canTravel(farePaise)) {
            balancePaise -= farePaise;
```

```
            cout << "Journey accepted." << endl;
        } else {
            cout << "Journey denied. Minimum balance is Rs.50." << endl;
        }
    }

    void displayBalance() {
        cout << "Card ID: " << cardId << endl;
        cout << "Balance: Rs." << balancePaise / 100
             << "." << (balancePaise % 100 < 10 ? "0" : "")
             << balancePaise % 100 << endl;
    }
};

int main() {
    SmartCard card("SC101", 10000); // Rs.100
    card.displayBalance();
    card.deductFare(2000);          // Rs.20
    card.displayBalance();
    return 0;
}
```

Expected output:



Explanation :

a class is a blueprint and card is an object. cardId and balancePaise are data members. deductFare() is a member function. balancePaise is private, so it cannot be changed directly outside the class.

Complexity

Time: O(1) for one balance check/deduction.

Extra space: O(1).

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly 3 Marks Bloom's Level: K5 – Evaluate

Define the fares, surcharge and limits so that a revision is made in one

place only. Discuss:

• The five kinds of tokens in your program (keywords, identifiers, constants, operators, punctuators) with examples

• #define, const and constexpr constants, and why constexpr is preferred for the fare table

• enum and enum class for zones, and why enum class is safer

• Naming rules and conventions for identifiers

Alternative Concept and Complexity Analysis

If the fares were written with #define macros instead of constexpr constants and an enum class, what would change? Compare type checking and scope, and analyse the memory used (a macro occupies no storage; a constexpr value usually does not either) and whether the running time is affected.

Evidence Expected

A single, well-named set of constants with a justified choice between #define, const, constexpr and enum class.

ANS:

Task02 – Use tokens, keywords, identifiers and constants correctly

Aim: To implement and demonstrate the task using a simple C++ program.

Complete code

```
#include <iostream>
using namespace std;

int main() {
    const int INNER_FARE = 1000;       // Rs.10 in paise
    const int MIDDLE_FARE = 2000;      // Rs.20 in paise
    const int OUTER_FARE = 3000;       // Rs.30 in paise
    const int PEAK_PERCENT = 20;
    const int MIN_BALANCE = 5000;      // Rs.50 in paise

    enum class Zone { Inner, Middle, Outer };
```

```
    Zone passengerZone = Zone::Middle;
    int fare = 0;

    switch (passengerZone) {
        case Zone::Inner:
            fare = INNER_FARE;
            break;
        case Zone::Middle:
            fare = MIDDLE_FARE;
            break;
        case Zone::Outer:
            fare = OUTER_FARE;
            break;
    }

    cout << "Middle zone base fare: Rs." << fare / 100 << endl;
    cout << "Peak surcharge: " << PEAK_PERCENT << "%" << endl;
    cout << "Minimum balance: Rs." << MIN_BALANCE / 100 << endl;
    return 0;
}
```

Expected output:



Explanation:

const values are fixed during the program. enum class Zone creates named zone choices. switch selects the matching base fare. Keywords include const, int, enum, class, switch and return.

Complexity

Time: O(1). Extra space: O(1).

TASK 3 – Choose Data Types and Check Type Compatibility 4 Marks

Bloom's Level: K5 – Evaluate

Select a C++ data type for every value in the system.

Discuss:

• The type chosen (bool, char, int, long long, unsigned, float, double, auto) for fares, balances, times, zone numbers and journey counts, with sizeof on your system
• Why money should be stored in integer paise and not as double rupees

• Type compatibility rules, and what the compiler does with int, double and bool values in the same expression

• The ranges from <climits> and whether int is enough for a day's collection across the network

Alternative Concept and Complexity Analysis

If money were stored as double rupees instead of long long paise, what would the solution be? Analyse the time per addition (O(1) in both), the space per value (8 bytes in both), and measure how the rounding error grows when n fares are added.

Evidence Expected

A justified data-type table with sizes and ranges, and evidence of one precision or overflow problem with its resolution.

ANS:
Aim: To implement and demonstrate the task using a simple C++ program.

Complete code

```
#include <iostream>
#include <climits>
#include <limits>
using namespace std;

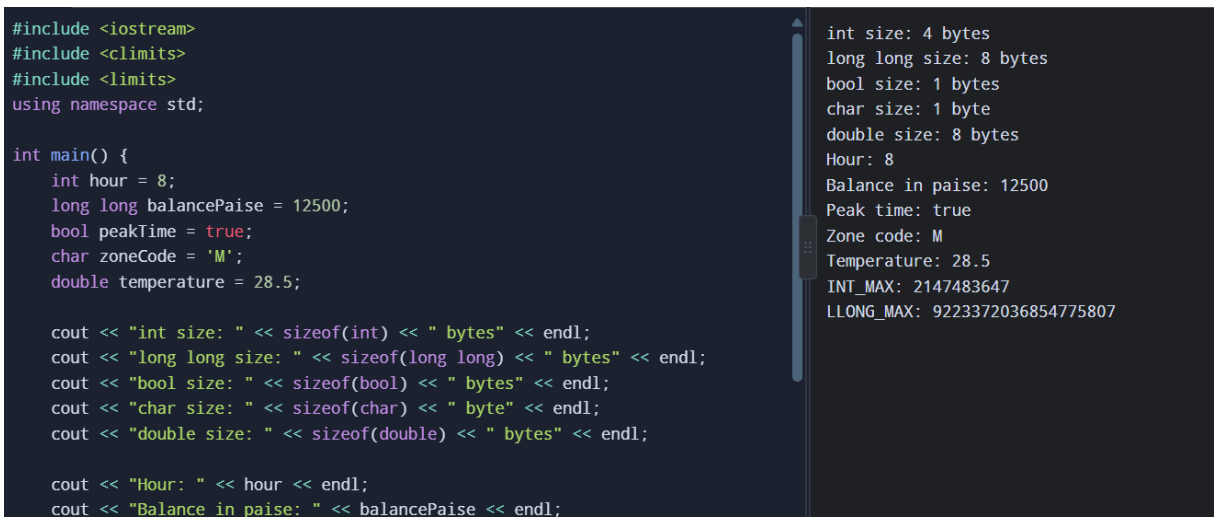
int main() {
    int hour = 8;
    long long balancePaise = 12500;
    bool peakTime = true;
    char zoneCode = 'M';
    double temperature = 28.5;
```

```
    cout << "int size: " << sizeof(int) << " bytes" << endl;
    cout << "long long size: " << sizeof(long long) << " bytes" << endl;
    cout << "bool size: " << sizeof(bool) << " bytes" << endl;
    cout << "char size: " << sizeof(char) << " byte" << endl;
    cout << "double size: " << sizeof(double) << " bytes" << endl;

    cout << "Hour: " << hour << endl;
    cout << "Balance in paise: " << balancePaise << endl;
    cout << "Peak time: " << boolalpha << peakTime << endl;
    cout << "Zone code: " << zoneCode << endl;
    cout << "Temperature: " << temperature << endl;

    cout << "INT_MAX: " << INT_MAX << endl;
    cout << "LLONG_MAX: " << LLONG_MAX << endl;
    return 0;
}
```

Expected output:



Explanation:

Use int for hour, long long for money totals, bool for a true/false condition, char for a single zone letter and double for measurements. Record the actual sizes printed by your own compiler; they are implementation-dependent.

Complexity

Time: O(1). Extra space: O(1).

TASK 4 – Control Implicit Conversions 4 Marks

Bloom's Level: K5 – Evaluate

Identify every place in the program where C++ converts a value from one type to another without being told.

Discuss:

• Integer division and integer-to-double conversion

• Comparison of signed and unsigned values, and the compiler warning it

produces • Narrowing conversions, and how brace initialisation (int x{3.7};)

prevents them • static_cast and when an explicit cast is needed

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()
{
 int a = 7, b = 2;
 double c = a / b;
 double d = static_cast<double>(a) / b;
 std::cout << c << ' ' << d << '\n';

 char ch = 'A' + 2;
 std::cout << ch << ' ' << ch + 1 << '\n';

 bool flag = 5;
 std::cout << flag << ' ' << std::boolalpha << flag << '\n';

 unsigned int u = 1;
 int s = -1;
 std::cout << (s < u ? "s < u" : "s >= u") << '\n';
```

}

My predicted output (write before running):

```
3 3.5
C 68
1 true
signedValue >= unsignedValue
```

Explain each line of the actual output, naming the conversion that took place (integer division, char to int promotion, int to bool, signed to unsigned).

Alternative Concept and Complexity Analysis

If every conversion in the program were written explicitly with static_cast (and brace initialisation were used everywhere), instead of relying on implicit conversions, what would the solution be? Analyse whether the running time or memory changes, and which errors are caught at compile time instead of at run time.

Evidence Expected

Identified conversion points with demonstrated surprises, and corrected code using brace initialisation or static_cast.

ANS:

Aim: To implement and demonstrate the task using a simple C++ program.

Complete code

```
#include <iostream>
using namespace std;

int main() {
   int a = 7, b = 2;

   double normalDivision = a / b;
   double correctDivision = static_cast<double>(a) / b;

   cout << normalDivision << " " << correctDivision << endl;

   char letter = 'A' + 2;
   cout << letter << " " << static_cast<int>(letter) + 1 << endl;

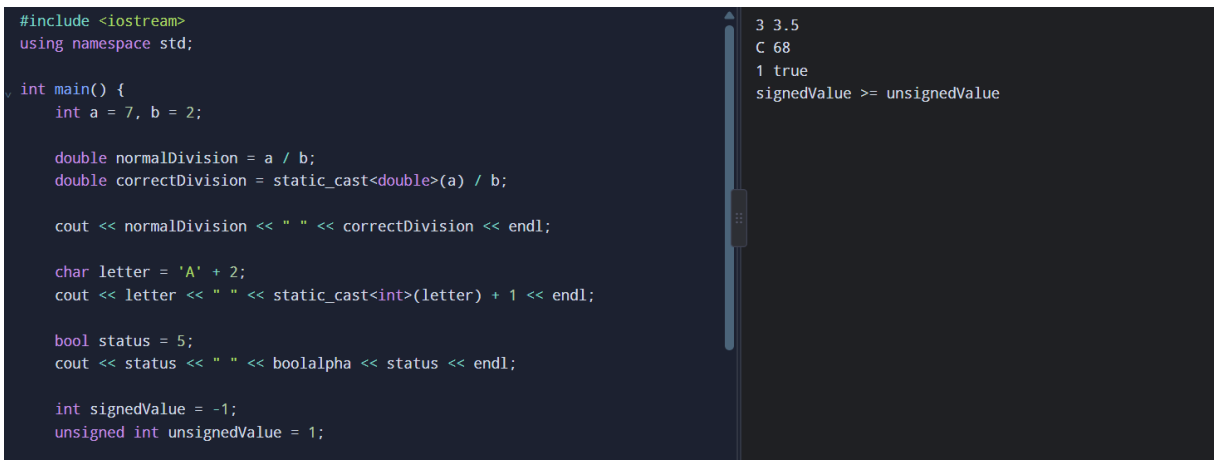
   bool status = 5;
   cout << status << " " << boolalpha << status << endl;

   int signedValue = -1;
   unsigned int unsignedValue = 1;
```

```
    if (signedValue < unsignedValue)
        cout << "signedValue < unsignedValue" << endl;
    else
        cout << "signedValue >= unsignedValue" << endl;

    return 0;
}
```

Expected output:



Explanation:

7/2 is integer division, so it gives 3 before conversion to double. static_cast<double>(a)/b gives 3.5. A char is promoted to int in arithmetic. Integer 5 converts to bool true. In the signed/unsigned comparison, -1 is converted to unsigned, so it is not less than 1.

Complexity

Time: O(1). Extra space: O(1).

TASK 5 – Write Correct Expressions with the Right Precedence 4 Marks

Bloom's Level: K5 – Evaluate
Write the expressions for the peak check, the surcharge and the concession.

Discuss:

• Operator precedence and associativity in each expression, including the conditional operator ?: and compound assignment

• The difference between x++ and ++x, and why i = i++ is a mistake

• The result of / and % with negative operands

• Why (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent but harder to read

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


int main()

{

 int x = 5, y = 3;

 std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';


 int z = x++ + ++y;

 std::cout << z << ' ' << x << ' ' << y << '\n';


 std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';

 std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';


 int k = 0;

 for (int i = 0; i < 10; i += 3) k += i;

 std::cout << k << '\n';

}
```

My predicted output (write before running):
For every line, name the operators involved and the order in which precedence and associativity apply them.

Alternative Concept and Complexity Analysis

If the peak-hour check were done with a lookup table bool peak[24] indexed by the hour, instead of a combined && / || expression, what would the solution be? Analyse the time per check (O(1) in both, but fewer comparisons with the table) and the extra space of 24 bytes.

Evidence Expected

Correct, readable expressions with an operator-by-operator explanation.

ANS:
Aim: To implement and demonstrate the task using a simple C++ program.

Complete code

```
#include <iostream>
using namespace std;

int main() {
   int x = 5, y = 3;

   cout << x + y * 2 << " " << (x + y) * 2 << endl;

   int result = x++ + ++y;
   cout << result << " " << x << " " << y << endl;

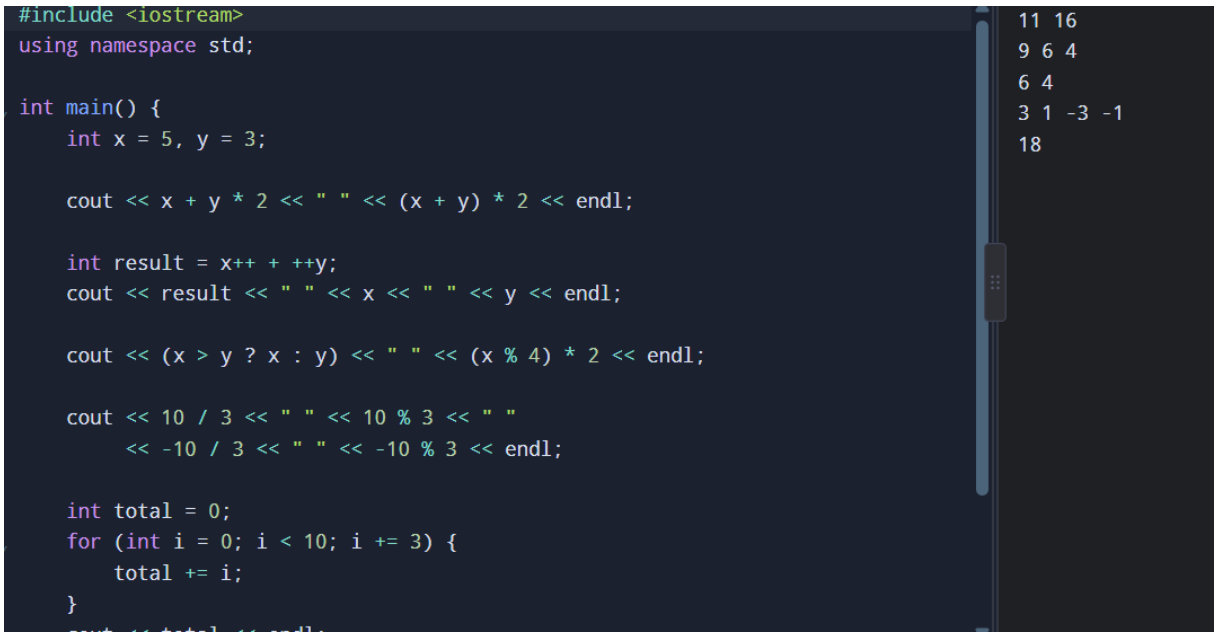
   cout << (x > y ? x : y) << " " << (x % 4) * 2 << endl;

   cout << 10 / 3 << " " << 10 % 3 << " "
        << -10 / 3 << " " << -10 % 3 << endl;

   int total = 0;
   for (int i = 0; i < 10; i += 3) {
      total += i;
   }
   cout << total << endl;

   return 0;
}
```

Expected output:



Explanation:

Multiplication is evaluated before addition. Parentheses change the order. x++ uses the old value and increments after; ++y increments before use. The ?: operator selects the larger value. Integer division truncates toward zero. The loop adds 0+3+6+9.

Complexity

Time: O(1) for the fixed expressions and loop. Extra space: O(1).

TASK 6 – Design the Control Structures 4 Marks

Bloom's Level: K5 – Evaluate

Design the decision and looping logic for computing a fare and processing a

day's journeys. Discuss:

• When to use if–else, a switch on the zone, and the conditional operator

• How a for loop, a range-based for loop and a while loop could process the journeys, and which is best

• Where break and continue are needed (for example, skipping an invalid

journey) • What happens when no case of a switch matches

Write pseudocode for the fare computation following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing C++ code.

Complete the Code

Complete TODO 1 and TODO 2 below using a switch statement and integer paise, and execute the program.

```
#include <iomanip>

#include <iostream>


enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs

20, Rs 30 constexpr int PEAK_SURCHARGE_PERCENT = 20;

constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card


long long fare(Zone z, bool peak)

{
```

```
 // TODO 1: Use a switch on z to pick the base fare from BASE_PAISE,
then add  // PEAK_SURCHARGE_PERCENT % in peak hours using
integer arithmetic only
}

bool canTravel(long long balance, long long f)
{
 // TODO 2: Return true only if the balance left after paying the fare is at
least  // MIN_BALANCE_PAISE

}

int main()
{
 long long balance = 10000; // Rs 100.00 in paise
 struct Trip { Zone zone; bool peak; };
 Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true},
{Zone::Middle, false}};  for (const Trip& t : trips) {
 long long f = fare(t.zone, t.peak);
 if (canTravel(balance, f)) {
 balance -= f;
 std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
 << std::setfill('0') << f % 100 << std::setfill(' ');
 } else {
 std::cout << "Denied (low balance)";
 }
 std::cout << " balance Rs " << balance / 100 << '\n';
 }
}
```

Expected output:

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

Add a journey at exactly 10 AM and at exactly 8 PM and check that your peak rule treats them correctly.

Alternative Concept and Complexity Analysis

If the base fare were looked up from an array indexed by the zone (constexpr long long FARE[] = {1000, 2000, 3000}) instead of a switch, what would the solution be? Analyse the time per lookup (O(1) in both; the switch may become a jump table), the space of the array, and how each design copes with a fourth zone.

Evidence Expected

Pseudocode and C++ control structures for the fare computation, with justified choices.

ANS:
Aim: To implement and demonstrate the task using a simple C++ program.

Complete code

```
#include <iostream>
#include <iomanip>
using namespace std;

enum class Zone { Inner, Middle, Outer };

const int INNER_FARE = 1000;
const int MIDDLE_FARE = 2000;
const int OUTER_FARE = 3000;
const int PEAK_PERCENT = 20;
const long long MIN_BALANCE = 5000;

long long calculateFare(Zone zone, bool peak) {
    long long fare = 0;

    switch (zone) {
        case Zone::Inner:  fare = INNER_FARE; break;
        case Zone::Middle: fare = MIDDLE_FARE; break;
        case Zone::Outer:  fare = OUTER_FARE; break;
    }
```

```
    if (peak) {
        fare = fare + (fare * PEAK_PERCENT / 100);
    }
    return fare;
}

bool canTravel(long long balance, long long fare) {
    return fare >= 0 && balance - fare >= MIN_BALANCE;
}

int main() {
    long long balance = 10000; // Rs.100

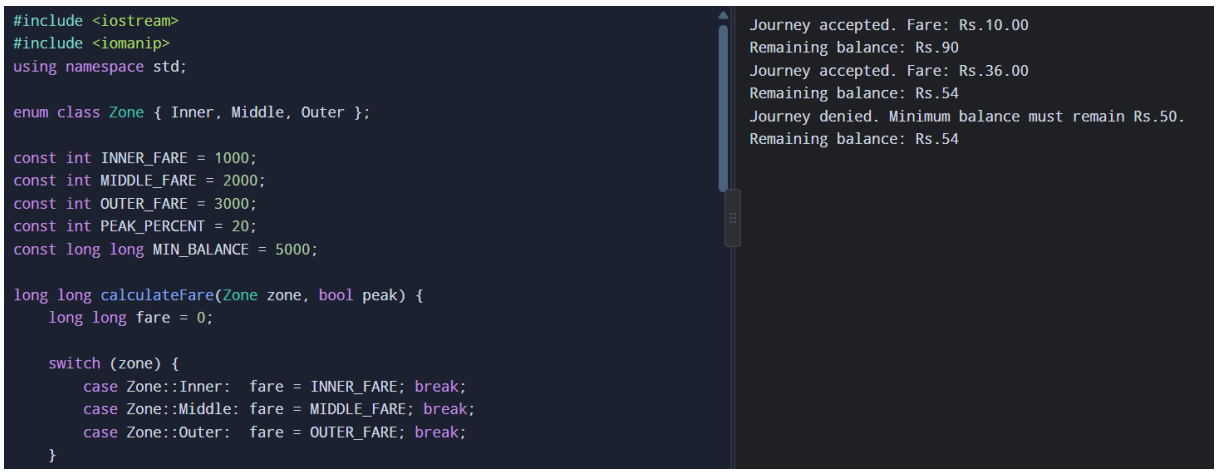
    Zone trips[] = {Zone::Inner, Zone::Outer, Zone::Middle};
    bool peak[] = {false, true, false};

    for (int i = 0; i < 3; i++) {
        long long fare = calculateFare(trips[i], peak[i]);

        if (canTravel(balance, fare)) {
            balance = balance - fare;
            cout << "Journey accepted. Fare: Rs."
                 << fare / 100 << "." << (fare % 100 < 10 ? "0" : "")
                 << fare % 100 << endl;
        } else {
            cout << "Journey denied. Minimum balance must remain Rs.50." << endl;
        }

        cout << "Remaining balance: Rs." << balance / 100 << endl;
    }
    return 0;
}
```

Expected output:



```
#include <iostream>
#include <iomanip>
using namespace std;

enum class Zone { Inner, Middle, Outer };

const int INNER_FARE = 1000;
const int MIDDLE_FARE = 2000;
const int OUTER_FARE = 3000;
const int PEAK_PERCENT = 20;
const long long MIN_BALANCE = 5000;

long long calculateFare(Zone zone, bool peak) {
    long long fare = 0;

    switch (zone) {
        case Zone::Inner:  fare = INNER_FARE; break;
        case Zone::Middle: fare = MIDDLE_FARE; break;
        case Zone::Outer:  fare = OUTER_FARE; break;
    }
```

```
Journey accepted. Fare: Rs.10.00
Remaining balance: Rs.90
Journey accepted. Fare: Rs.36.00
Remaining balance: Rs.54
Journey denied. Minimum balance must remain Rs.50.
Remaining balance: Rs.54
```

Explanation:

switch chooses the base fare. if checks the peak surcharge and minimum balance. for repeats the same decision for three trips. A denied journey does not deduct money.

Complexity

Time: O(n) for n trips. Extra space: O(n) for the fixed trip arrays; O(1) per iteration.

TASK 7 – From Procedural to Object-Oriented 4 Marks

Bloom's Level: K5 – Evaluate

Rewrite the fare logic in two styles.

Discuss:

• A procedural version with separate variables and free functions

• An object-oriented version with a SmartCard class that keeps its balance private and offers deduct() and canTravel()

• How the class prevents a negative balance

• Which version is easier to change when the minimum balance changes

Alternative Concept and Complexity Analysis

If the balance were kept in a public data member instead of a private member with deduct() and canTravel(), what would the solution be? Analyse whether time or space changes (they do not), and explain which rule of the system can then be broken by any part of the program.

Evidence Expected

Two working versions and a reasoned comparison of the procedural and object-oriented designs.

ANS:

Aim: To implement and demonstrate the task using a simple C++ program.

Complete code

```
#include <iostream>
using namespace std;

// Procedural approach
bool proceduralCanTravel(long long balance, long long fare) {
    return fare >= 0 && balance - fare >= 5000;
}

// Object-oriented approach
class SmartCard {
private:
    long long balancePaise;

public:
    SmartCard(long long initialBalance) {
        balancePaise = initialBalance;
    }

    bool canTravel(long long fare) {
        return fare >= 0 && balancePaise - fare >= 5000;
    }

    bool deductFare(long long fare) {
        if (!canTravel(fare)) {
            return false;
        }
        balancePaise = balancePaise - fare;
        return true;
    }

    long long getBalance() {
        return balancePaise;
    }
};
```

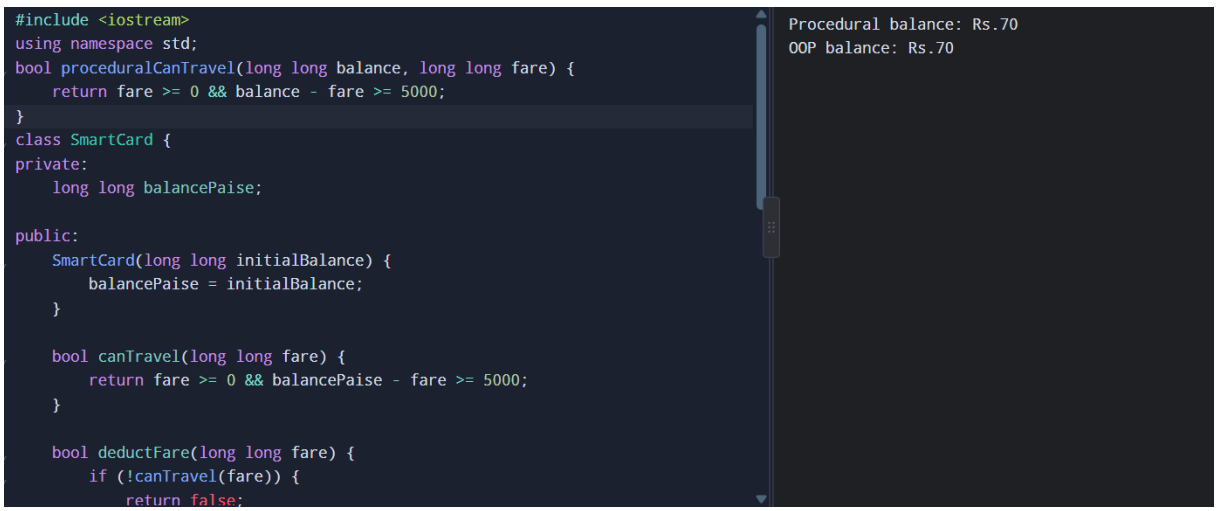
```
int main() {
    long long proceduralBalance = 10000;
    long long fare = 3000;

    if (proceduralCanTravel(proceduralBalance, fare)) {
        proceduralBalance -= fare;
        cout << "Procedural balance: Rs." << proceduralBalance / 100 << endl;
    } else {
        cout << "Procedural journey denied." << endl;
    }

    SmartCard card(10000);
    if (card.deductFare(fare)) {
        cout << "OOP balance: Rs." << card.getBalance() / 100 << endl;
    } else {
        cout << "OOP journey denied." << endl;
    }

    return 0;
}
```

Expected output:



Explanation:

The procedural version uses separate functions and a balance variable. The OOP version stores balance inside SmartCard. private prevents direct outside modification. Both apply the same minimum-balance rule.

Complexity

Time: O(1) for one journey. Extra space: O(1) per card.

TASK 8 – Read and Print Data Safely 4 Marks

Bloom's Level: K5 – Evaluate

Design the input and output of the gate program.

Discuss:

• Reading values with cin and checking cin.fail() when the operator types letters instead of numbers

• Clearing the error state with cin.clear() and discarding the bad input

• Formatting money with <iomanip> (setw, setfill, fixed, setprecision)

• Printing a neat journey report

Alternative Concept and Complexity Analysis

If each input line were read with std::getline() and converted with std::stoi() inside try/catch, instead of reading numbers directly with cin >>, what would the solution be? Analyse the time to process a line of length L (O(L) in both) and the buffer space needed, and compare how each recovers from invalid input.

Evidence Expected

Safe input handling with demonstrated recovery from invalid input, and correctly formatted output.

ANS:

Aim: To implement and demonstrate the task using a simple C++ program.

Complete code

```
#include <iostream>
#include <iomanip>
#include <limits>
using namespace std;

int main() {
    int hour;

    cout << "Enter journey hour (0 to 23): ";
```

```
  while (!(cin >> hour) || hour < 0 || hour > 23) {
     cout << "Invalid input. Enter a whole number from 0 to 23: ";
     cin.clear();
     cin.ignore(numeric_limits<streamsize>::max(), '\n');
  }

  cout << "Accepted hour: " << hour << endl;
  cout << fixed << setprecision(2);
  cout << "Example fare: Rs." << 10.5 << endl;

  return 0;
}
```

Expected output:



Explanation:

cin reads the integer. When abc is entered, extraction fails. clear() resets the error flag and ignore() removes the invalid line. The loop asks again until a valid hour is entered.

Complexity

Time: O(k) where k is the number of attempts. Extra space: O(1).

TASK 9 – Implement and Test the Fare Logic 4 Marks

Bloom's Level: K5 – Evaluate

Implement the complete fare logic and test it.

Test:

• Normal journeys in each zone, peak and off-peak

• Boundary cases: exactly 8:00, 10:00, 17:00 and 20:00; a balance that leaves exactly Rs

50 • Invalid cases: an unknown zone, a negative recharge, letters typed for the hour • A small set of 5 journeys whose total you calculate by hand

Alternative Concept and Complexity Analysis

If each journey were tested against a table of test cases (inputs and expected fares) in a loop, instead of separate hand-written checks, what would the solution be? Analyse the time for t tests (O(t)) and the space of the table, and how easily a new boundary case can be added.

Evidence Expected

Working source code and a test table with expected and actual results for every case.

ANS:

Aim: To implement and demonstrate the task using a simple C++ program.

Input / execution steps

Enter the sample input below. Each journey line contains zone number and hour.

```
5
1 7
1 8
2 10
3 17
3 20
```

Complete code

```
#include <iostream>
#include <iomanip>
using namespace std;

enum class Zone { Inner = 1, Middle = 2, Outer = 3 };

const long long MIN_BALANCE = 5000;
const int PEAK_PERCENT = 20;

long long getBaseFare(Zone zone) {
    switch (zone) {
        case Zone::Inner: return 1000;
        case Zone::Middle: return 2000;
        case Zone::Outer: return 3000;
    }
    return -1;
}
```

```
bool isPeakHour(int hour) {
    return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}

long long calculateFare(Zone zone, int hour) {
    long long fare = getBaseFare(zone);

    if (fare < 0 || hour < 0 || hour > 23) {
        return -1;
    }

    if (isPeakHour(hour)) {
        fare = fare + (fare * PEAK_PERCENT / 100);
    }
    return fare;
}

int main() {
    long long balance = 20000; // Rs.200
    int numberOfTrips;

    cout << "Enter number of journeys: ";
    if (!(cin >> numberOfTrips) || numberOfTrips < 1 || numberOfTrips > 20) {
        cout << "Invalid journey count." << endl;
        return 1;
    }

    for (int i = 1; i <= numberOfTrips; i++) {
        int zoneNumber, hour;

        cout << "Journey " << i << " - Enter zone (1=Inner, 2=Middle, 3=Outer) and hour: ";
        if (!(cin >> zoneNumber >> hour)) {
            cout << "Invalid input. Program stopped." << endl;
            return 1;
        }

        if (zoneNumber < 1 || zoneNumber > 3 || hour < 0 || hour > 23) {
            cout << "Invalid journey. Balance unchanged." << endl;
            continue;
        }

        Zone zone = static_cast<Zone>(zoneNumber);
```

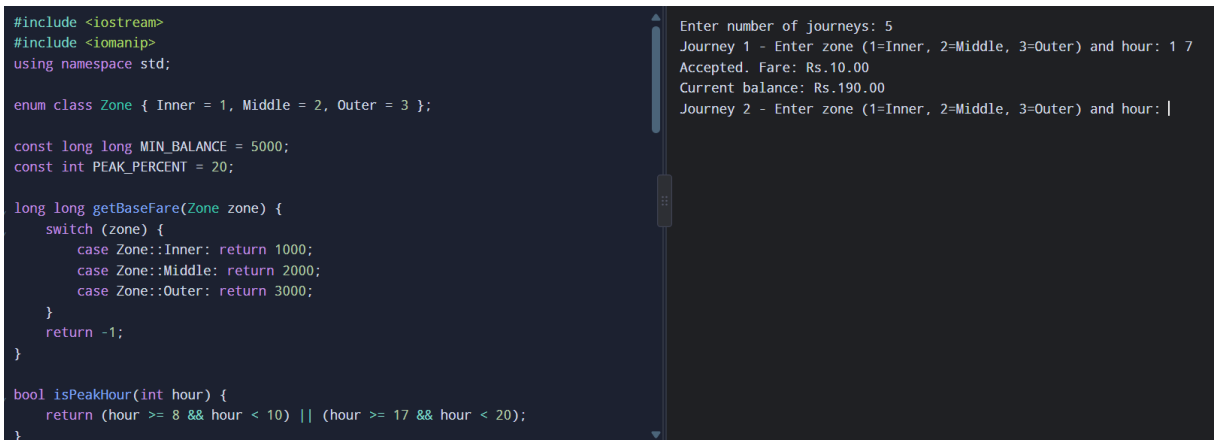
```
    long long fare = calculateFare(zone, hour);

    if (balance - fare >= MIN_BALANCE) {
        balance -= fare;
        cout << "Accepted. Fare: Rs." << fare / 100 << "."
             << (fare % 100 < 10 ? "0" : "") << fare % 100 << endl;
    } else {
        cout << "Denied. Minimum balance of Rs.50 must remain." << endl;
    }

    cout << "Current balance: Rs." << balance / 100 << "."
         << (balance % 100 < 10 ? "0" : "") << balance % 100 << endl;
  }

  return 0;
}
```

Expected output:



Explanation:

Zone 1 is Inner, 2 is Middle, 3 is Outer. Hours 8 and 17 are peak; 10 and 20 are off-peak. The program validates the zone/hour before calculating. It deducts only if at least Rs.50 remains.

Complexity

Time: O(n) for n journeys. Extra space: O(1), because journeys are processed one at a time.

TASK 10 – Compare Expectations with Execution 4 Marks

Bloom's Level: K5 – Evaluate

Compare what you expected with what actually happened.

Discuss:

• Whether the double and integer-paise totals for 20,000, 500,000 and 3,000,000 journeys agree, and by how much they differ

• Whether int overflows for the network total

• The compiler warnings you received with -Wall -Wextra and what each

meant • Reasons for any difference

Alternative Concept and Complexity Analysis

If the network total were accumulated in an int, a long long or a double, what would happen at 3,000,000 journeys? Analyse when each type fails (overflow versus loss of precision), and compare the time per addition and the space per variable.

Evidence Expected

A reasoned comparison of expected and observed behaviour, supported by measured totals.

ANS:

Aim: To calculate total fare for increasing journey counts and explain why long long is needed.

Complete code

```
#include <iostream>
#include <iomanip>
#include <climits>
using namespace std;

int main() {
    long long journeyCounts[] = {20000, 500000, 3000000};

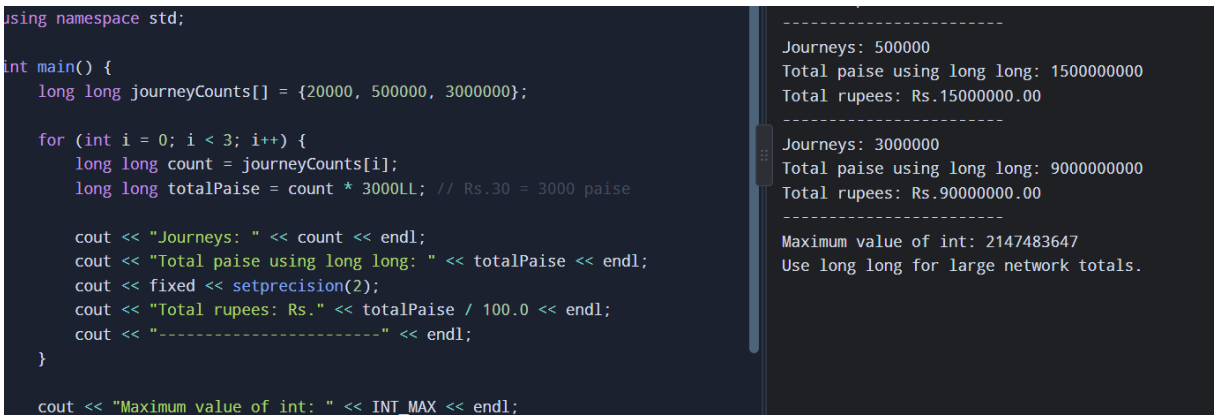
    for (int i = 0; i < 3; i++) {
        long long count = journeyCounts[i];
        long long totalPaise = count * 3000LL; // Rs.30 = 3000 paise

        cout << "Journeys: " << count << endl;
        cout << "Total paise using long long: " << totalPaise << endl;
        cout << fixed << setprecision(2);
        cout << "Total rupees: Rs." << totalPaise / 100.0 << endl;
        cout << "------------------------" << endl;
    }

    cout << "Maximum value of int: " << INT_MAX << endl;
    cout << "Use long long for large network totals." << endl;
```

```
    return 0;
}
```

Expected output:



```
using namespace std;

int main() {
    long long journeyCounts[] = {20000, 500000, 3000000};

    for (int i = 0; i < 3; i++) {
        long long count = journeyCounts[i];
        long long totalPaise = count * 3000LL; // Rs.30 = 3000 paise

        cout << "Journeys: " << count << endl;
        cout << "Total paise using long long: " << totalPaise << endl;
        cout << fixed << setprecision(2);
        cout << "Total rupees: Rs." << totalPaise / 100.0 << endl;
        cout << "------------------------" << endl;
    }

    cout << "Maximum value of int: " << INT_MAX << endl;
```

```
------------------------
Journeys: 500000
Total paise using long long: 1500000000
Total rupees: Rs.15000000.00
------------------------
Journeys: 3000000
Total paise using long long: 9000000000
Total rupees: Rs.90000000.00
------------------------
Maximum value of int: 2147483647
Use long long for large network totals.
```

Explanation: A single Rs.30 fare is 3000 paise. Three million journeys produce 9,000,000,000 paise, which is greater than the usual 32-bit int maximum 2,147,483,647. long long can store this total.
Complexity: O(1) because only three fixed totals are calculated.
Extra space O(1).

TASK 11 – Recommend and Defend the Final Design 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the final design for the metro.

Discuss:

• The types, constants and classes you recommend and why

• How a fare revision will be carried out

• The limitations of your design

• Whether the design remains correct for the whole network
Your recommendation must be based on evidence from your own

experiments. Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, instead of being compiled into the program, what would the solution be? Analyse the extra start-up time to read z fare values and the space they occupy, against the cost of recompiling and redeploying after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

ANS:

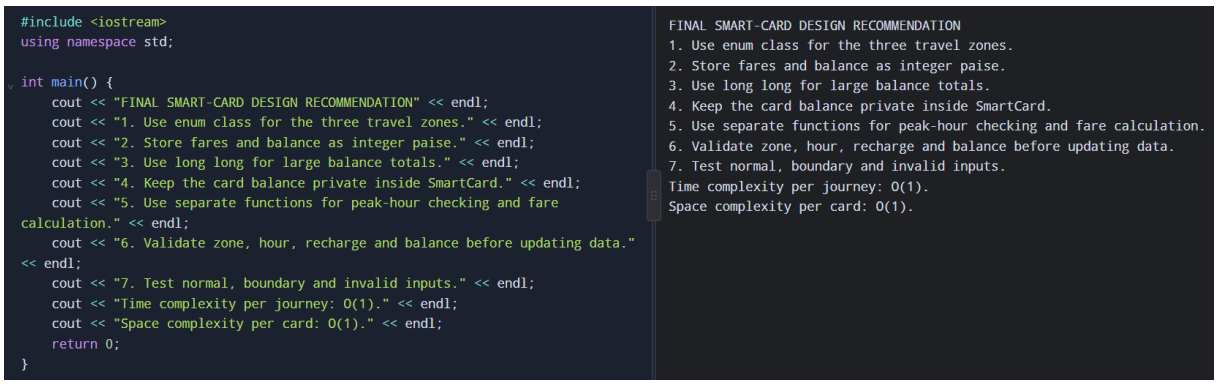
Aim: To present the final recommended design decisions in an executable summary.
Input: No input is required.

Complete code

```
#include <iostream>
using namespace std;

int main() {
    cout << "FINAL SMART-CARD DESIGN RECOMMENDATION" << endl;
    cout << "1. Use enum class for the three travel zones." << endl;
    cout << "2. Store fares and balance as integer paise." << endl;
    cout << "3. Use long long for large balance totals." << endl;
    cout << "4. Keep the card balance private inside SmartCard." << endl;
    cout << "5. Use separate functions for peak-hour checking and fare calculation." << endl;
    cout << "6. Validate zone, hour, recharge and balance before updating data." << endl;
    cout << "7. Test normal, boundary and invalid inputs." << endl;
    cout << "Time complexity per journey: O(1)." << endl;
    cout << "Space complexity per card: O(1)." << endl;
    return 0;
}
```

Expected output:



Explain: These choices improve safety, readability, maintainability and accuracy. The balance is private, currency is integer paise, and validation occurs before a card is updated.

Complexity: O(1) time per journey and O(1) space per card.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones.

Re-evaluate your original design. You may RETAIN → MODIFY → or REDESIGN your

original solution. A. Impact Analysis – 2 Marks

Identify which constants, enum values, expressions, switch statements and classes are

affected. Evidence Expected

A clear analysis of how the changed requirements affect the original

design. B. Evaluation of the Original Design – 2 Marks

Evaluate whether the switch-based fare logic and the SmartCard class can absorb the changes, using worked examples.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe and implement the revised design (for example: a fare table indexed by zone, a separate discount rule, a Pass class).

Evidence Expected

An appropriate design response with working code.

D. Evidence-Based Defence – 2 Marks

Defend your decision using test results and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating tests and the changed requirements.

ANS:

Aim: To add an Airport zone, a weekend discount and a two-zone monthly pass.

Rules used

- Inner = Rs.10; Middle = Rs.20; Outer = Rs.30; Airport = Rs.60.
- Weekday peak surcharge = 20%.
- Weekend discount = 10%.
- Weekend discount takes priority and is not combined with peak surcharge.
- At least Rs.50 must remain after a stored-value payment.
- The sample MonthlyPass covers two selected zones.

Complete code

```
#include <iostream>
#include <array>
using namespace std;

enum class Zone { Inner = 0, Middle = 1, Outer = 2, Airport = 3 };

const array<long long, 4> BASE_FARES = {1000, 2000, 3000, 6000};
const long long MIN_BALANCE = 5000;

long long calculateFare(Zone zone, int hour, bool weekend) {
    int index = static_cast<int>(zone);

    if (index < 0 || index >= 4 || hour < 0 || hour > 23) {
        return -1;
    }

    long long fare = BASE_FARES[index];
    bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);

    // Weekend discount takes priority; it is not combined with peak surcharge.
    if (weekend) {
        fare = fare - (fare * 10 / 100);
    } else if (peak) {
        fare = fare + (fare * 20 / 100);
    }

    return fare;
}

class SmartCard {
private:
    long long balancePaise;

public:
    SmartCard(long long balance) {
        balancePaise = balance;
    }

    bool pay(long long fare) {
        if (fare < 0 || balancePaise - fare < MIN_BALANCE) {
            return false;
        }
```

```
        balancePaise -= fare;
        return true;
    }

    long long getBalance() {
        return balancePaise;
    }
};

class MonthlyPass {
private:
    Zone firstZone;
    Zone secondZone;
    bool active;

public:
    MonthlyPass(Zone first, Zone second) {
        firstZone = first;
        secondZone = second;
        active = true;
    }

    bool covers(Zone zone) {
        return active && (zone == firstZone || zone == secondZone);
    }
};

int main() {
    cout << "Airport weekday off-peak: " << calculateFare(Zone::Airport, 12, false) << "
paise" << endl;
    cout << "Airport weekday peak: " << calculateFare(Zone::Airport, 8, false) << " paise"
<< endl;
    cout << "Airport weekend off-peak: " << calculateFare(Zone::Airport, 12, true) << "
paise" << endl;
    cout << "Airport weekend peak: " << calculateFare(Zone::Airport, 8, true) << " paise" <<
endl;

    MonthlyPass pass(Zone::Inner, Zone::Middle);
    cout << boolalpha;
    cout << "Pass covers Inner zone: " << pass.covers(Zone::Inner) << endl;
    cout << "Pass covers Airport zone: " << pass.covers(Zone::Airport) << endl;

    SmartCard card(10000);
```

```
    cout << "Payment successful: " << card.pay(3000) << endl;
    cout << "Remaining card balance: " << card.getBalance() << " paise" << endl;

    return 0;
}
```

Expected output:



Explanation:

The fare table has four entries. The program checks the zone and hour first. If it is a weekend, it applies 10% discount. Otherwise, if it is peak time, it applies 20% surcharge. This if/else order prevents combining the two rules. MonthlyPass checks whether a requested zone is one of its two covered zones. SmartCard keeps the balance private.

Complexity

Fare calculation is O(1), card payment is O(1), and pass coverage is O(1). The four-value fare table uses O(1) space.

Test table

Test	Input	Expected fare / result	Actual result
Airport weekday off-peak	Zone Airport, 12:00	Rs.60.00	Fill after run
Airport weekday peak	Zone Airport, 08:00	Rs.72.00	Fill after run
Airport weekend off-peak	Weekend, 12:00	Rs.54.00	Fill after run
Airport weekend peak	Weekend, 08:00	Rs.54.00; no combination	Fill after run
Pass covered zone	Inner	true	Fill after run
Pass uncovered zone	Airport	false	Fill after run

MARK DISTRIBUTION

Task Bloom's Level Marks

Task 1 – Object-Oriented Analysis K5 – Evaluate 4

Task 2 – Tokens and Constants K5 – Evaluate 3

Task 3 – Data Types and Compatibility K5 – Evaluate 4

Task 4 – Implicit Conversions K5 – Evaluate 4

Task 5 – Operators and Precedence K5 – Evaluate 4

Task 6 – Control Structures K5 – Evaluate 4

Task 7 – Procedural vs Object-Oriented K5 – Evaluate 4

Task 8 – Input and Output K5 – Evaluate 4

Task 9 – Implementation & Testing K5 – Evaluate 4

Task 10 – Expectation vs Execution K5 – Evaluate 4

Task 11 – Final Recommendation K5 – Evaluate 4

Subtotal 42

Task 12 – Day-4 Design Challenge K5 – Evaluate 8

TOTAL 50

Submission Requirements

• GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.

• Your assignment report containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.